

# Appendix: SOLID Principles

Miguel de Jesús Palacios Rivera

SOLID stands for:

- **Single Responsibility Principle:** A class should have only one reason to change. Each class should have a single task to do, and that responsibility should be well-done. **Example:** A class that handles user authentication should not also handle user data storage.
- **Open/Closed Principle:** Software entities (classes, modules, functions, etc.) should be open for extension but closed for modification. This means that the behavior of a module can be extended without modifying its source code. **Example:** Using inheritance or interfaces on a payment processing system to add new payment methods without changing the existing code. You should use an interface named `PaymentMethod` and then create new classes that implement this interface for each new payment method.
- **Liskov Substitution Principle:** Objects of a superclass should be replaceable with objects of its subclasses without affecting the correctness of the program. **Example:** If you have the class `Bird` and a subclass `Penguin`, you should be able to use a `Penguin` object wherever a `Bird` object is expected without causing errors or unexpected behavior.
- **Interface Segregation Principle:** Clients should not be forced to depend on interfaces they do not use. You should build several smaller, more specific interfaces instead of one large interface. **Example:** Instead of having a single `Vehicle` interface with methods for driving, flying, and sailing, you could have separate interfaces for `Drivable`, `Flyable`, and `Sailable`. This way, a class that only needs to drive doesn't have to implement methods for flying or sailing.
- **Dependency Inversion Principle:** High-level modules should not depend on low-level modules. Both should depend on abstractions. Additionally, abstractions should not depend on details. Details should depend on abstractions. **Example:** Using dependency injection to decouple components and make them more testable and maintainable.